

Sintaxis de Lenguajes de Programación

2025

Enfoque del tema: Sintaxis de LP

□ **Introducción a la definición de lenguajes**

Importancia

Sintaxis y Semántica

□ **Conceptos de Sintaxis**

Propósito y Criterios generales

Elementos Sintácticos de un Lenguaje

□ **Especificación de la sintaxis**

Estructura Léxica

Gramáticas libres de contexto

Notación BNF y EBNF

Arboles sintácticos

Diagramas de sintaxis

Ambigüedad. Gramáticas ambiguas

Definición de un Lenguaje I

“Un lenguaje de programación es una notación formal para describir algoritmos a ser ejecutados por computadoras”.

- **Sintaxis**

Forma de las expresiones, sentencias y unidades de programa

- **Semántica**

Significado de expresiones, sentencias y unidades de programa

Definición de un Lenguaje II

La *definición de un lenguaje* permite determinar:

- Si un programa es válido.
- Cuál es su significado o efecto.

Ejemplo: Sentencia condicional en C

Sintaxis: `if (<expresión>) <sentencia>`

Semántica: "Si el valor actual de la expresión es cierto, se ejecuta la sentencia siguiente"

Conceptos de Sintaxis

- Sintaxis

Conjunto de reglas y criterios de escritura que permiten la formación de programas correctos en un lenguaje, considerando sólo el punto de vista de la representación

- ✓ *Las reglas especifican la composición de programas a partir de letras, dígitos y otros caracteres.*
- ✓ *A partir de estas reglas puede establecerse si una sentencia es legal o no.*

- Reglas léxicas
- Reglas sintácticas

Reglas

- **Reglas léxicas:** Definen el conjunto de caracteres que constituyen el alfabeto del lenguaje y la forma de combinar dichos caracteres para formar palabras válidas. Por ejemplo, Java y Python consideran en forma diferente las letras mayúsculas y minúsculas.
- **Reglas sintácticas:** Definen cómo pueden formarse las sentencias a partir de constituyentes básicos llamados *palabras*.

- **Propósito**

Proveer una notación para la comunicación entre el programador y el procesador de lenguajes de programación.

- **Criteria Sintácticos**

- legibilidad,
- facilidad de escritura,
- facilidad de verificación,
- facilidad de traducción,
- carencia de ambigüedad

Elementos Sintácticos de un LP

- Caracteres
- Identificadores
- Operadores
- Palabras clave y reservadas
- Comentarios
- Espacios en blanco
- Delimitadores y corchetes
- Expresiones
- Sentencias o Enunciados

Especificación de Sintaxis

- La definición formal de la sintaxis de un LP se conoce como gramática
- La gramática se compone de un conjunto de reglas que especifican la serie de caracteres y cadenas que forman programas permisibles en el LP que se está definiendo
- La especificación de la sintaxis considera:
 - Estructura léxica de un LP
 - Gramática formal definida en alguna notación

Estructura Léxica

- **Lenguaje:** conjunto de cadenas de caracteres pertenecientes a algún alfabeto.
- **Sentencia:** cada una de estas cadenas.
- **Lexemas:** unidades sintácticas de más bajo nivel. Incluyen: identificadores operadores y palabras especiales
- **Token:** categoría de lexemas. El lexema es atributo del token.

• Ejemplo Lexemas y Tokens

Sentencia en C:

```
indice = 5 * contador + 1;
```

Lexema Token

indice	identificador
=	op-asignación
5	constante_entera
*	op_producto
contador	identificador
+	op_suma
12	constante_entera
;	delimitador

Representa una descripción formal de la sintaxis de un lenguaje

Se especifica usando una notación definida de manera estricta

FORTRAN: Definido con algunas reglas en inglés

Python: Descripción formal de su sintaxis con BNF (gramática libre de contexto), un metalenguaje para definir lenguajes de programación.

Gramática libre de contexto

Clasificación de Chomski (1959): gramáticas regulares y **gramáticas libres de contexto**



Conjunto de reglas que definen todas las construcciones válidas que puede aceptar un lenguaje.

Formalmente (N, T, S, P) :

- N o conjunto de símbolos no terminales.
- T o conjunto de símbolos terminales.
- S o símbolo inicial
- P o conjunto de reglas de producción.

Metalenguaje BNF

- **Métodos comunes de descripción de sintaxis**

Gramática formal Backus-Naur (BNF) y EBNF

Diagramas o grafos sintácticos

- **Símbolos del metalenguaje (metasímbolos)**

BNF (Backus Naur Form)

::=	"se define como"
.	"fin de una definición"
	"or lógico, alternativa"
<>	"encierran los no terminales"

Los terminales se escriben tal y como son

* "cero o más ocurrencias del elemento precedente"

+ "una o más ocurrencias del elemento previo"

BNF utiliza *abstracciones para definir las estructuras sintácticas* .

Un enunciado de asignación de C, por ejemplo, puede ser representado por la abstracción <assign>.

Los brackets "< >" se utilizan para delimitar nombres de abstracciones.

La definición de <assing> puede darse por:

<assign> ::= <var> = < expression>

o de la forma

<assign> → <var> = < expression>

Metalenguaje BNF

- ✓ El símbolo del lado izquierdo de la flecha, el cual se llama el lado izquierdo (LHS), *es la abstracción que se está definiendo.*
- ✓ El texto del lado derecho de la flecha (RHS) *es la definición del LHS.*
- ✓ En conjunto, la definición es llamada "regla" o "producción".

La regla ejemplo especifica: "la abstracción <assign> es definida como una instancia de la abstracción <var> seguida del lexema "=", seguida por una instancia de la abstracción <expression>."

Una sentencia ejemplo cuya estructura sintáctica se describe por la regla es: total = sum1 + sum2

Metalinguaje BNF

Las abstracciones en una gramática BNF, se denominan símbolos no terminales y los lexemas y tokens de las reglas se llaman símbolos terminales.

Los símbolos no terminales pueden tener dos o más definiciones distintas, representando dos o más formas posibles en el lenguaje. Por ejemplo, un enunciado if de Pascal puede ser descripto con las reglas:

```
<enunc_if> ::= if <expr_log> then <enunc>  
<enunc_if> ::= if <expr_log> then <enunc> else <enunc>
```

o utilizando OR con el símbolo "|", se describe con la regla:

```
<enunc_if> ::= if <expr_log> then <enunc> | if <expr_log> then  
<enunc> else <enunc>
```

Gramática para un Lenguaje simple

Las gramáticas pueden emplearse como **generadoras** de programas válidos.

Comenzando en el símbolo inicial y sustituyendo en cada paso un símbolo no terminal por su definición hasta que todos los símbolos son terminales.

Ejemplo de Gramática para un LP simple

Lenguaje definido por una gramática: conjunto de cadenas de componentes léxicos **derivadas** del símbolo inicial de la GRT

• Lenguaje con única sentencia de asignación

<programa>	→	begin <lista_sentencias> end
<lista_sentencias>	→	<sentencia>
		<sentencia> ; <lista_sentencias>
<sentencia>	→	<var> := <expresión>
<var>	→	A B C D
<expresión>	→	<var> + <var>
		<var> - <var>
		<var>

Las gramáticas pueden emplearse como generadores de programas válidos en el Lenguaje

Ejemplo de Gramática para un LP simple

- ✓ Este lenguaje tiene una única sentencia de asignación.
- ✓ El programa consiste de una palabra especial **begin** seguida por una lista de sentencias separadas por punto y comas, seguida por la palabra especial **end**.
- ✓ Una expresión es tanto una variable simple o dos variables o también un operador + o -. Los únicos nombres de variables son A, B, C o D.

Generación de un Programa válido

La gramática se describe mostrando la lista de sus producciones

• Derivación en el lenguaje definido

```
<programa>  ⇒ begin <lista_sentencias> end
              ⇒ begin <sentencia> ; <lista_sentencias> end
              ⇒ begin <var> := <expresión> ; <lista_sentencias>
                  end
              ⇒ begin B := <expresión> ; <lista_sentencias> end
              ⇒ begin B := <var> - <var> ; <lista_sentencias> end
              ⇒ begin B := B - <var> ; <lista_sentencias> end
              ⇒ begin B := B - C ; <lista_sentencias> end
              ⇒ begin B := B - C ; <sentencia> end
              ⇒ begin B := B - C ; <var> := <expresión> end
              ⇒ begin B := B - C ; A := <expresión> end
              ⇒ begin B := B - C ; A := <var> end
              ⇒ begin B := B - C ; A := D end
```

Generación de un Programa válido

- ✓ La derivación comienza con el símbolo de comienzo, en este caso `<program>`.
- ✓ El símbolo " $\Rightarrow$ " se lee "deriva".
- ✓ Cada string sucesivo en la secuencia se deriva del string anterior, reemplazando uno de los no-terminales con una de sus definiciones.
- ✓ Cada uno de los strings en la derivación, incluyendo `<program>` se llama "forma de sentencia" (sentential form).
- ✓ La derivación continúa hasta que la forma no contiene más no-terminales.

Arboles sintácticos

- Representan las estructuras sintácticas jerárquicas de sentencias de un LP definidas en la gramática

Es un árbol etiquetado en el cual:

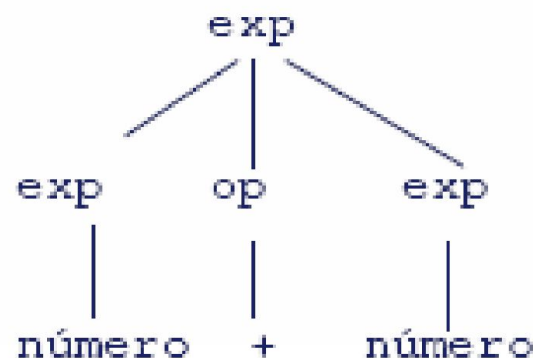
- los nodos interiores están etiquetados por no terminales,
- los nodos hoja están etiquetados por terminales y
- los hijos de cada nodo interno representan el reemplazo del no terminal asociado en un paso de la derivación

`exp` $\rightarrow$ `exp op exp`

`exp` $\rightarrow$ `numero`

`op` $\rightarrow$ `+`

`op` $\rightarrow$ `-`



Gramática para sentencia simple de asignación

El ejemplo que se muestra es de una gramática para una parte de un lenguaje de programación típico.

$$\langle \text{assign} \rangle ::= \langle \text{id} \rangle := \langle \text{expr} \rangle$$
$$\langle \text{id} \rangle ::= A \mid B \mid C$$
$$\langle \text{expr} \rangle ::= \langle \text{id} \rangle + \langle \text{expr} \rangle \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle *$$

La gramática del ejemplo describe sentencias de asignación cuyos lados derechos son expresiones aritméticas con operadores de adición, multiplicación y paréntesis.

Gramática para un Lenguaje simple

$\langle \text{assign} \rangle ::= \langle \text{id} \rangle := \langle \text{expr} \rangle$

$\langle \text{id} \rangle ::= A \mid B \mid C$

$\langle \text{expr} \rangle ::= \langle \text{id} \rangle + \langle \text{expr} \rangle \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

A := B * (A + C) es generada por la derivación:

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle := \langle \text{expr} \rangle$

$\Rightarrow A := \langle \text{expr} \rangle$

$\Rightarrow A := \langle \text{id} \rangle * \langle \text{expr} \rangle$

$\Rightarrow A := B * \langle \text{expr} \rangle$

$\Rightarrow A := B * (\langle \text{expr} \rangle)$

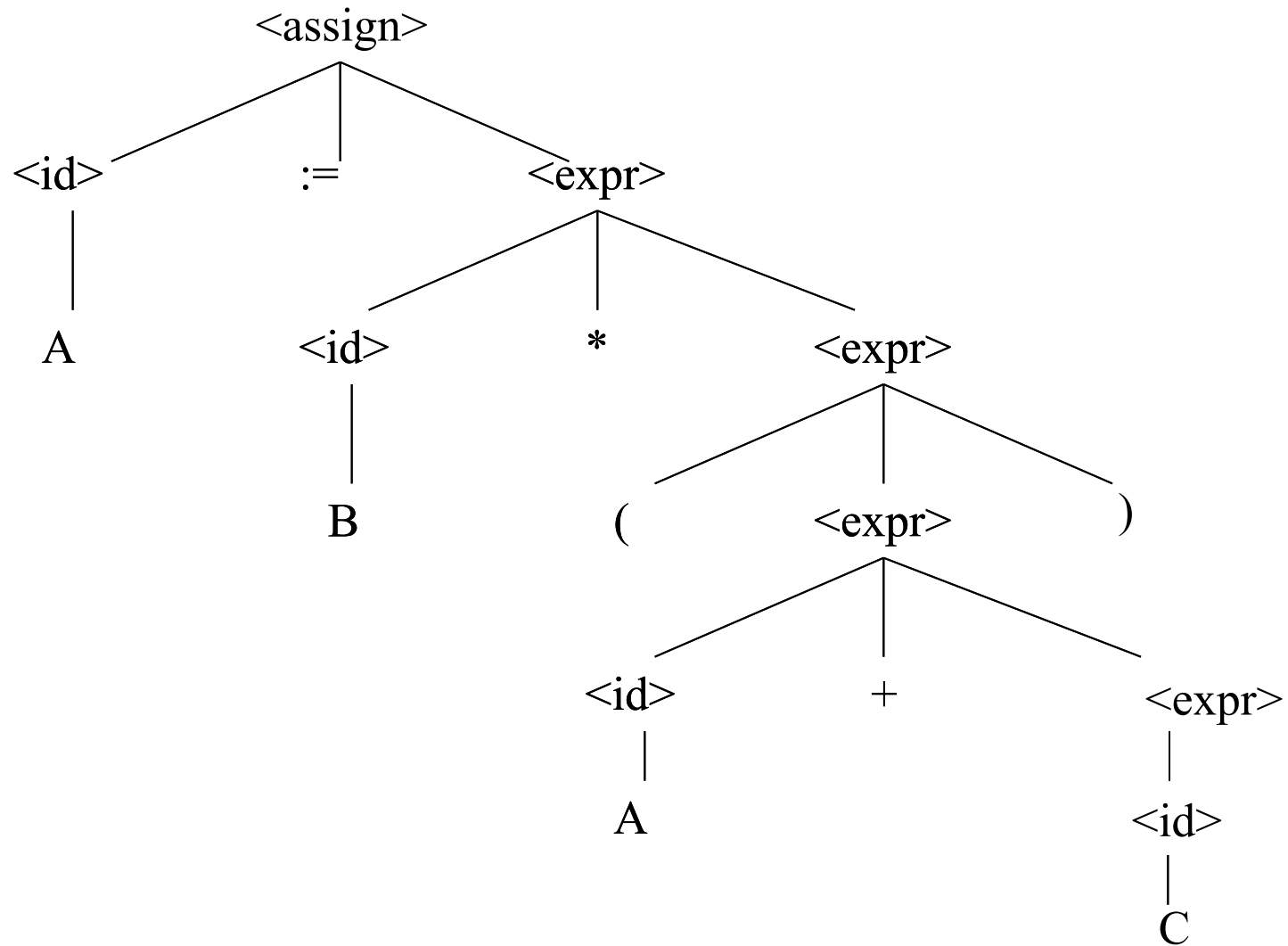
$\Rightarrow A := B * (\langle \text{id} \rangle + \langle \text{expr} \rangle)$

$\Rightarrow A := B * (A + \langle \text{expr} \rangle)$

$\Rightarrow A := B * (A + \langle \text{id} \rangle)$

$\Rightarrow A := B * (A + C)$

Estructura de derivación para el ejemplo



Dada la siguiente definición BNF de la sintaxis de una sentencia de asignación

$\langle \text{sent_asig} \rangle ::= \langle \text{var} \rangle := \langle \text{expresion} \rangle$
 $\langle \text{expresion} \rangle ::= \langle \text{expresion} \rangle + \langle \text{termino} \rangle \mid \langle \text{expresion} \rangle - \langle \text{termino} \rangle \mid \langle \text{termino} \rangle$
 $\langle \text{termino} \rangle ::= \langle \text{termino} \rangle * \langle \text{factor} \rangle \mid \langle \text{termino} \rangle / \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$
 $\langle \text{factor} \rangle ::= (\langle \text{expresion} \rangle) \mid \langle \text{var} \rangle \mid \langle \text{num} \rangle$
 $\langle \text{var} \rangle ::= A \mid B \mid C \mid D \mid \dots \mid Z$
 $\langle \text{num} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Encuentra la derivación que genera la siguiente expresión. Dibuja el árbol de derivación correspondiente.

$C := D - E * F$

Gramáticas Ambiguas

Una gramática es ambigua si permite construir dos o más árboles de derivación distintos para la misma cadena.

Una gramática en la cual, toda cadena generada por la misma, todas las derivaciones tienen el mismo árbol de derivación, no es ambigua.

La siguiente es una definición BNF simplificada para sentencias de asignación de un lenguaje tipo Pascal:

$\langle \text{sent_asig} \rangle ::= \langle \text{var} \rangle := \langle \text{expresión} \rangle$

$\langle \text{expresión} \rangle ::= \langle \text{expresión} \rangle + \langle \text{expresión} \rangle \mid \langle \text{expresión} \rangle - \langle \text{expresión} \rangle \mid (\langle \text{expresión} \rangle) \mid$
 $\langle \text{expresión} \rangle * \langle \text{expresión} \rangle \mid \langle \text{expresión} \rangle / \langle \text{expresión} \rangle \mid \langle \text{var} \rangle \mid \langle \text{num} \rangle$

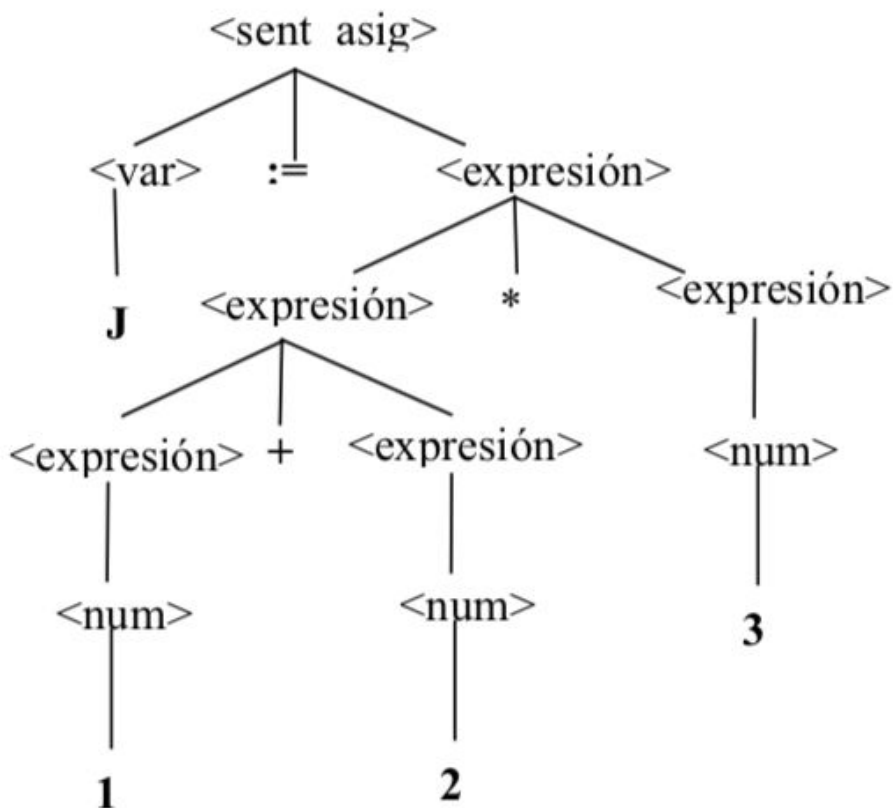
$\langle \text{var} \rangle ::= A \mid B \mid C \mid D \mid \dots \mid Z$

$\langle \text{num} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

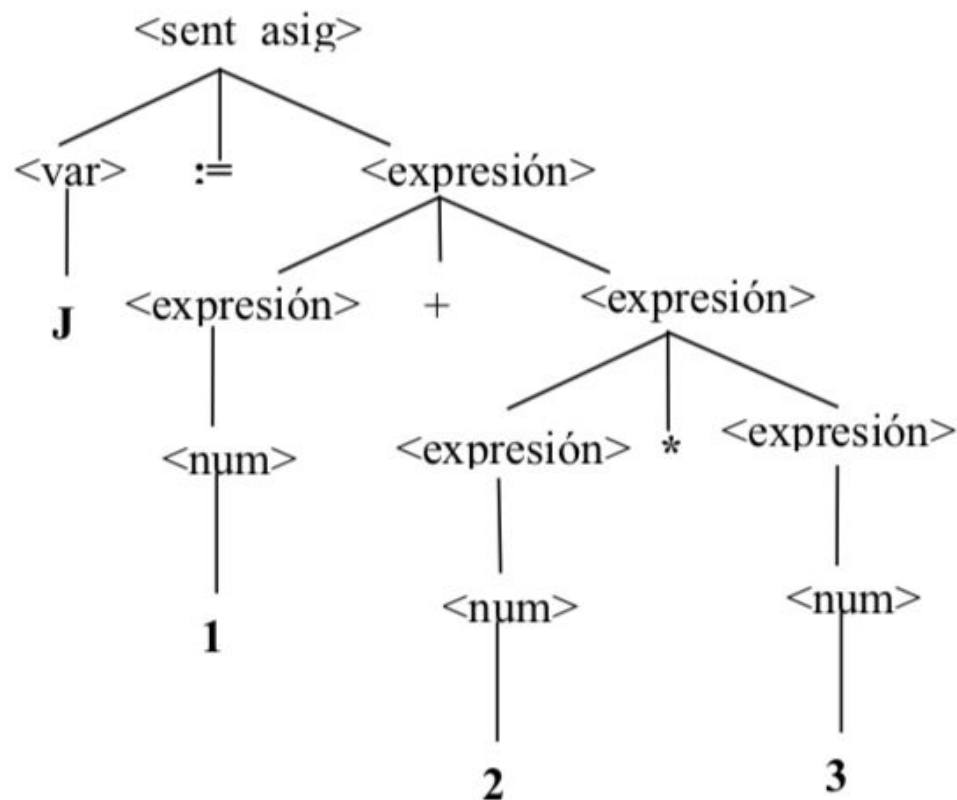
Gramáticas Ambiguas

Para la cadena $J := 1 + 2 * 3$ existen dos árboles de derivación posibles.

Árbol 1



Árbol 2



EBNF (BNF Extendido)

• Notación EBNF

[] indica opcionalidad en la parte derecha de la regla

Ej: $\langle \text{valor} \rangle \rightarrow [\langle \text{signo} \rangle] \langle \text{núm_sin_signo} \rangle [. \text{núm_sin_signo}]$

- { } indica repetición en la parte derecha de la regla

Ej: $\langle \text{programa} \rangle ::= \{ \langle \text{sentencia} \rangle + \}$

- | indica alternativa en la parte derecha de la regla encerrado entre corchetes si es necesario

Ej: $\langle \text{identificador} \rangle ::= \langle \text{letra} \rangle | \langle \text{identificador} \rangle [\langle \text{letra} \rangle | \langle \text{dígito} \rangle]$

Definición EBNF para un LP simple

Reglas sintácticas

$\langle \text{programa} \rangle ::= \{ \langle \text{sentencia} \rangle^* \}$

$\langle \text{sentencia} \rangle ::= \langle \text{asignación} \rangle \mid \langle \text{condicional} \rangle \mid \langle \text{loop} \rangle$

$\langle \text{asignación} \rangle ::= \langle \text{identificador} \rangle = \langle \text{expr} \rangle$

$\langle \text{condicional} \rangle ::= \text{if} \langle \text{expr} \rangle \{ \langle \text{sentencia} \rangle^* \} \mid$

$\text{if} \langle \text{expr} \rangle \{ \langle \text{sentencia} \rangle^* \} \text{ else } \{ \langle \text{sentencia} \rangle^* \}$

$\langle \text{loop} \rangle ::= \text{while} \langle \text{expr} \rangle \{ \langle \text{sentencia} \rangle^* \}$

$\langle \text{expr} \rangle ::= \langle \text{identificador} \rangle \mid \langle \text{numero} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{expr} \rangle \langle \text{operador} \rangle \langle \text{expr} \rangle$

Reglas léxicas

$\langle \text{operador} \rangle ::= + \mid - \mid * \mid / \mid = \mid \neq \mid < \mid > \mid \leq \mid \geq$

$\langle \text{identificador} \rangle ::= \langle \text{letra} \rangle \langle \text{id} \rangle^*$

$\langle \text{id} \rangle ::= \langle \text{letra} \rangle \mid \langle \text{digito} \rangle$

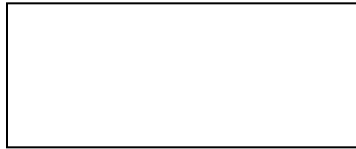
$\langle \text{numero} \rangle ::= \langle \text{digito} \rangle^*$

$\langle \text{letra} \rangle ::= a \mid b \mid c \mid \dots \mid z$

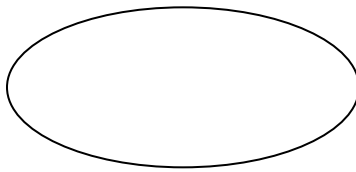
$\langle \text{digito} \rangle ::= 0 \mid 1 \mid \dots \mid 9$

Diagrama de sintaxis

Tipo de notación para especificar en forma gráfica la sintaxis de un Lenguaje. Se usan líneas y figuras en lugar de nombres



Símbolo no terminal



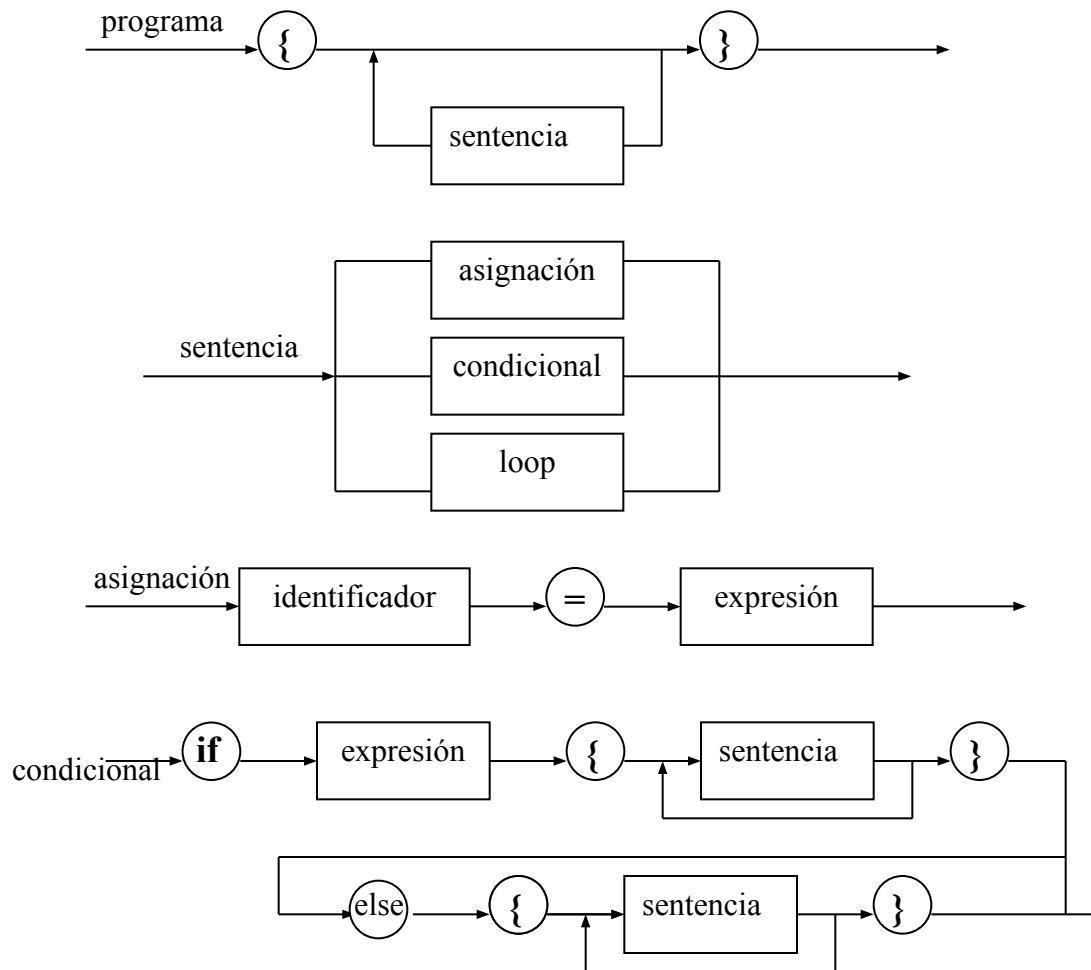
Símbolo terminal



Definición de un
Símbolo no terminal

Cada regla se representa por un camino que va desde la entrada a la izquierda del DG a la salida a la derecha

Ejemplo Diagrama de sintaxis



Continúa

Diagrama de sintaxis

- Una cadena de palabras es un programa válido, si puede generarse atravesando el diagrama de sintaxis desde el borde de entrada al borde de salida.
- Si se encuentra un terminal (círculo), esa palabra debe estar en la cadena siendo reconocida;
- Si se encuentra un noterminal (caja), entonces ese noterminal debe ser reconocido atravesando el diagrama de transición para dicho noterminal.
- Cuando se encuentra una ramificación, cada borde debe ser atravesado.

Utilización de la descripción sintáctica

- **En síntesis, dos usos principales:**

Ayuda al programador a saber cómo escribir un programa sintácticamente correcto.

Se utiliza para determinar cuándo un programa es sintácticamente correcto, que es lo que hace el compilador.

Bibliografía

- Sebesta R., [Concepts of Programming Languages](#), Addison Wesley, 2019
- Ghezzi C., [Programming Languages Concepts](#), John Wiley & sons, 2000
- Aaby A., [Introduction to Programming Languages](#), Electronic ed., 2003
- Sethi R., [Lenguajes de Programación. Conceptos y Construcciones](#), Addison Wesley, 1992
- Pratt T., [Lenguajes de Programación. Diseño e implementación 3ª ed.](#), Prentice Hall, 1998
- Urciuolo A., [Apuntes de clase Cátedra Lenguajes de Programación](#), UNPSJB, 2003